CHAPTER- 1

INTRODUCTION

1.1 EDGE ARTIFICIAL INTELLIGENCE (EDGE AI)

Edge Artificial Intelligence (Edge AI) refers to the deployment of AI and machine learning algorithms directly on edge devices—such as sensors, smartphones, cameras, drones, and embedded systems—rather than relying solely on centralized cloud servers. These devices are equipped with computing capabilities that allow them to collect, process, and analyse data locally, enabling real-time decision-making without constant internet connectivity or human intervention. For learners and aspiring developers, working on Edge AI projects is increasingly important, as it combines skills from AI, embedded systems, and software development into practical, real-world applications.

Edge AI enhances traditional data processing by reducing latency, minimizing bandwidth usage, and improving system responsiveness. By processing data at the source, only relevant or summarized information is transmitted to the cloud, making the system more efficient and scalable. It often works alongside technologies such as machine learning, deep learning, and computer vision to enable applications like facial recognition, predictive maintenance, autonomous vehicles, and smart healthcare systems.

However, Edge AI systems also face significant challenges, particularly in terms of security and resource constraints. Since edge devices often operate in uncontrolled or remote environments and handle sensitive data—such as video feeds, biometric information, or industrial data—they are vulnerable to cyberattacks and data breaches. Additionally, edge devices typically have limited processing power, memory, and energy, which restricts the use of complex AI models and heavy security mechanisms. Therefore, designing lightweight, efficient, and secure AI models is a critical concern in Edge AI, ensuring data privacy, reliability, and trust while maintaining high performance.

1.2 OVERVIEW

Vision is one of the most essential human senses, shaping the way people understand, interpret, and interact with the world around them. It provides access to a constant stream of information about objects, shapes, movement, colours, distance, and spatial layout. This information enables individuals to perform daily activities such as navigating a room, identifying household items, reading labels, understanding written instructions, recognizing people, and responding to environmental changes. For someone with full vision, these tasks require almost no conscious effort. However, for individuals who are visually challenged, even simple activities can become complex and time-consuming, requiring extra concentration, assistance, or workaround methods. The absence or reduction of visual perception has a significant influence on a person's lifestyle, mobility, safety, independence, and social participation.

The global impact of visual impairment is substantial. According to the World Health Organization, hundreds of millions of people live with moderate, severe, or complete vision loss. Among them, nearly 40 million individuals are completely blind, and many more have low vision that cannot be corrected through glasses or surgery. Visual impairment is more prevalent in low-income and developing countries, where early intervention and medical care are often limited. People with vision loss rely heavily on alternative sensory cues such as touch and sound, but these senses cannot fully replace the richness and speed of information that vision normally provides. As a result, visually challenged individuals often experience difficulties in independent mobility, communication, education, and professional activities. These limitations can lead to reduced participation in society and can affect personal confidence and overall quality of life.

Traditional tools for visually challenged individuals—such as walking canes, tactile floor paths, Braille books, and human guides—play an important role in physical mobility and accessing certain types of information. While these tools are helpful, they cannot interpret complex scenes, identify multiple objects, or extract detailed visual information. For instance, a walking cane can detect obstacles but cannot identify what the object is, read a signboard, or recognize an item on a table. Human guides can provide detailed descriptions, but relying on another person reduces independence. Furthermore, access to trained guides or specialized support services is limited in many parts of the world. As technology continues to advance, there is a growing need for

innovative solutions that can assist visually challenged individuals in obtaining visual information in a more convenient and independent manner.

In the last decade, rapid advancements in computer vision and artificial intelligence have significantly influenced assistive technology research. Modern deep learning models can detect objects, recognize scenes, identify patterns, and extract textual information from images with remarkable accuracy. Additionally, embedded computing platforms such as the Raspberry Pi have evolved to become more powerful while maintaining low cost, small size, and energy efficiency. These developments have encouraged researchers to explore new ways of integrating AI-based vision capabilities into portable devices that can be used by visually challenged individuals. The goal is to create tools that can interpret visual information and translate it into a form that the user can understand and act upon.

Despite these technological advancements, existing commercial visual assistance systems have several limitations. Many high-end devices depend heavily on cloud servers for processing images and analysing scenes. While cloud-based computation allows for the use of large and powerful AI models, it also introduces challenges such as slow response time, dependency on stable internet connection, privacy concerns, and high operational costs. Devices that rely solely on offline processing often sacrifice accuracy or functionality due to hardware limitations. High-end wearable devices and smart glasses available in the market are often expensive, limiting their adoption among users in low-income communities. Many existing systems also offer only a single feature—such as text reading or navigation assistance—rather than combining multiple capabilities into one unit. These gaps highlight the need for a more accessible, multi-functional system that combines affordability, reliability, and practical usefulness.

The project titled **Edge AI Enabled Smart Vision Assistant for Visually Challenged Individuals** is developed with the aim of addressing these existing gaps. The system is built using the Raspberry Pi 4 microcomputer, which provides sufficient processing power to run lightweight AI models while maintaining portability. A camera module attached to the Raspberry Pi captures real-time images of the user's surroundings. For object detection, the system employs the **MobileNet-SSD deep learning model**, which is specifically designed for efficient performance on embedded devices. MobileNet-SSD is well-known for its small size, fast inference speed, and reliable accuracy in detecting common objects. Unlike large and computationally heavy models,

MobileNet-SSD performs real-time object detection without requiring specialized hardware such as GPUs.

In addition to object detection, the system integrates Optical Character Recognition (OCR) using the Tesseract OCR engine. This component allows the system to analyze printed text from documents, labels, signboards, and other materials. OCR capability is crucial for visually challenged individuals because written text appears in almost every environment—shops, public spaces, schools, offices, and homes. By enabling the device to extract text and interpret its content, the system broadens its applicability and helps users access information that would otherwise be inaccessible without assistance. The combination of object detection and text recognition provides a foundation for a versatile and practical visual assistance tool.

The system is designed to use internet connectivity for additional support when required. Although the core functions—MobileNet-SSD object detection and OCR—run directly on the Raspberry Pi, the device can connect to cloud services for updates, higher-level processing tasks, or additional modules. This hybrid approach makes the system more flexible and future-ready. Cloud connectivity enables developers to update AI models, push improvements, or add new features without requiring users to replace the hardware. This approach also ensures that the system can evolve with advancements in AI research, making it more adaptable and long-lasting.

Several technologies work together to form the complete system. Edge computing allows most operations to be executed locally, minimizing latency and ensuring timely information extraction. Computer vision techniques provide the ability to interpret visuals and identify different types of objects. OCR technology serves as a bridge between the visual world and printed text. The combination of these components is integrated into a streamlined pipeline that continuously captures images, analyses them, and produces meaningful results. This integrated approach allows the device to function in various environments such as homes, classrooms, streets, supermarkets, and workplaces.

Affordability remains one of the central considerations in the design of this project. Many advanced assistive devices in the market cost hundreds or thousands of dollars, making them inaccessible for a large portion of visually challenged individuals worldwide. By using low-cost hardware and open-source software packages, the project ensures that the device can be produced at a significantly lower cost. This makes the system suitable not just for individual users but also

for institutions such as schools for the visually impaired, rehabilitation centres, libraries, and non-profit organizations. The affordability and modularity of the design also encourage researchers and developers to build upon the system and customize it for specific use cases.

Another important aspect of the project is its potential for expansion. The system's architecture is designed in a modular way, allowing new features to be integrated without redesigning the entire device. Future enhancements could include advanced scene understanding, environmental mapping, gesture recognition, or integration with sensors for distance measurement. Since the system supports internet connectivity, any new capabilities or improvements can be deployed through software updates. This modular and upgradeable structure ensures that the system can grow over time and adapt to future technological innovations.

The combination of object detection and OCR in a single device allows the system to provide a wide range of visual information. Users can benefit from the system in different settings—identifying items in the kitchen, reading product labels in a shop, understanding printed instructions, recognizing items in a workspace, or interpreting signboards. Real-time performance is essential for the usefulness of such a device, especially in dynamic environments. MobileNet-SSD's ability to process images quickly ensures that the system provides timely interpretation of visual scenes.

Overall, the development of the Edge AI Enabled Smart Vision Assistant demonstrates how modern computer vision techniques and embedded technology can be combined to create practical solutions for visually challenged individuals. By focusing on object detection, OCR, efficient processing, affordability, and internet-based operation, the system lays the foundation for a scalable and accessible assistive tool. Its design reflects an effort to bring meaningful technological support to individuals who rely on alternative methods for accessing visual information. The system highlights the growing potential of AI-based visual assistance in helping visually challenged users navigate everyday environments with greater confidence and efficiency.

1.3 PROBLEM STATEMENT

Visually challenged individuals face difficulty in identifying objects and reading printed text in their surroundings, which limits independence. Existing assistive solutions are often costly or rely heavily on internet connectivity. Hence, there is a need for a low-cost, real-time Edge AI–based smart vision system that can assist visually impaired users by processing visual information locally.

1.4 OBJECTIVES

- **To develop a portable smart vision assistance device using edge AI**
  The primary objective of the project is to design and implement a compact and affordable vision-assistance system that can operate directly on embedded hardware. The device should be capable of performing essential visual processing tasks without depending entirely on external computing resources.
- **To implement real-time object detection using the MobileNet-SSD model**
  A key goal is to integrate the MobileNet-SSD deep learning model to detect commonly encountered objects in real time. The system should recognize objects accurately and process video frames efficiently on the Raspberry Pi, ensuring smooth performance for everyday use.
- **To enable text extraction using Optical Character Recognition (OCR)**
  Another objective is to incorporate OCR technology through Tesseract OCR to read printed text from items such as documents, labels, signboards, and product packaging. This allows the device to extract written information that visually challenged individuals cannot access.
- **To combine object detection and text recognition in a single integrated platform**
  The project aims to build a multi-purpose assistive system that offers both object recognition and text reading capabilities within one device, avoiding the need for multiple separate tools.

- **To ensure the system functions effectively with internet connectivity**

  Although the primary processing occurs on the device, the system should also support internet-based features when needed. This includes updates, cloud resources, and potential future expansions requiring online access.
- **To achieve low-latency processing through edge AI**

  An important objective is to minimize delays in object detection and text recognition. Running AI models on the Raspberry Pi ensures faster response times, making the system more practical for real-world use where timely information matters.
- **To design the system to be affordable and accessible**

  The goal is to use low-cost hardware and open-source software tools so that visually challenged individuals, schools, rehabilitation centres, and organizations can adopt the system without financial burden. Affordability is a core objective in making assistive technology more inclusive.
- **To create a user-friendly and simple operational workflow**

  The device should be easy to operate, with straightforward controls and minimal configuration. The objective is to ensure that users with visual challenges can use the system independently without requiring technical knowledge.

CHAPTER 2

LITERATURE SURVEY

In this paper [1], the author presented a comprehensive survey on assistive systems for visually impaired individuals. The study discusses various technologies such as sensor-based navigation, object recognition, and environment analysis, highlighting how intelligent systems can improve mobility and independence.

In this paper [2], the author reviewed both commercial and research-based assistive devices for visually challenged people. The focus was mainly on affordability, embedded processing, and the challenges faced in real-world deployment of such systems.

In this paper [3], the author explained advancements in smart wearable mobility aids. Camera-based assistive systems were discussed with emphasis on usability, comfort, and safety for visually impaired users.

In this paper [4], the author surveyed TinyML techniques for assistive and embedded vision applications. The paper highlights optimization techniques such as model quantization and pruning to enable AI deployment on low-power edge devices.

In this paper [5], the author discussed real-world deployment case studies of assistive vision applications in public spaces such as museums. The paper highlights practical challenges like latency, usability, and privacy.

In this paper [6], the author explained ethical considerations in AI-based accessibility tools. The study emphasizes privacy preservation, responsible AI design, and user consent in assistive technologies.

In this paper [7], the author examined recent trends in assistive tools for low-vision individuals. The evolution of camera-based systems and AI-enhanced solutions was discussed in detail.

In this paper [8], the author reviewed AI applications for accessibility in education. The study shows how AI-based assistive tools help visually impaired learners access educational content more effectively.

In this paper [9], the author benchmarked deep learning object detection models such as MobileNet-SSD, SSD-Lite, and Efficient Det-Lite on edge devices like Raspberry Pi. Performance metrics such as speed and accuracy were analysed.

In this paper [10], the author evaluated the performance of SSD-based object detection models on Raspberry Pi, focusing on latency, throughput, and computational efficiency.

In this paper [11], the author introduced the MobileNet architecture and its efficiency for mobile and embedded vision applications. This work forms the foundation for MobileNet-SSD used in edge devices.

In this paper [12], the author proposed the Single Shot Multibox Detector (SSD) architecture, which enables real-time object detection and is widely adopted in embedded systems.

In this paper [13], the author discussed optimization techniques for MobileNet-SSD, including quantization and pruning, to improve performance on edge platforms.

In this paper [14], the author presented a comparative study of lightweight object detectors on Raspberry Pi, showing that MobileNet-SSD offers a good balance between speed and accuracy.

In this paper [15], the author focused on real-time object detection using optimized SSD models under embedded system constraints.

In this paper [16], the author reviewed efficient CNN techniques such as MobileNet-v2 and MobileNet-v3 for edge-based object detection.

In this paper [17], the author evaluated lightweight SSD models for smart camera and IoT applications.

In this paper [18], the author proposed MobileNetV3-SSD for embedded vision, demonstrating improved performance and energy efficiency.

In this paper [19], the author reviewed low-power deep learning techniques for deploying object detection models on single-board computers.

In this paper [20], the author discussed quantization-aware training techniques to optimize SSD-based object detection models for edge AI systems.

In this paper [21], the author conducted a comprehensive study of open-source OCR libraries, comparing Tesseract and other OCR engines for embedded applications.

In this paper [22], the author evaluated OCR performance under noisy conditions such as low light and motion blur, highlighting robustness challenges.

In this paper [23], the author analysed the performance of Tesseract OCR with different image preprocessing techniques to improve text recognition accuracy.

In this paper [24], the author surveyed OCR engines suitable for mobile and embedded devices, focusing on lightweight and efficient implementations.

In this paper [25], the author discussed recent enhancements in Paddle OCR and other lightweight OCR frameworks for edge deployment.

In this paper [26], the author focused on scene text recognition optimized for low-resource hardware platforms.

In this paper [27], the author compared OCR systems for document images and natural scene text captured through cameras.

In this paper [28], the author reviewed embedded vision systems using Raspberry Pi and discussed performance limitations and optimization strategies.

In this paper [29], the author presented a systematic review of Edge AI deployment on single-board computers, comparing power consumption and latency.

In this paper [30], the author discussed case studies of vision-based wearable devices for visually challenged individuals, focusing on hardware design and form-factor considerations.

CHAPTER 3

PROPOSED WORK

3.1 BLOCK DIAGRAM

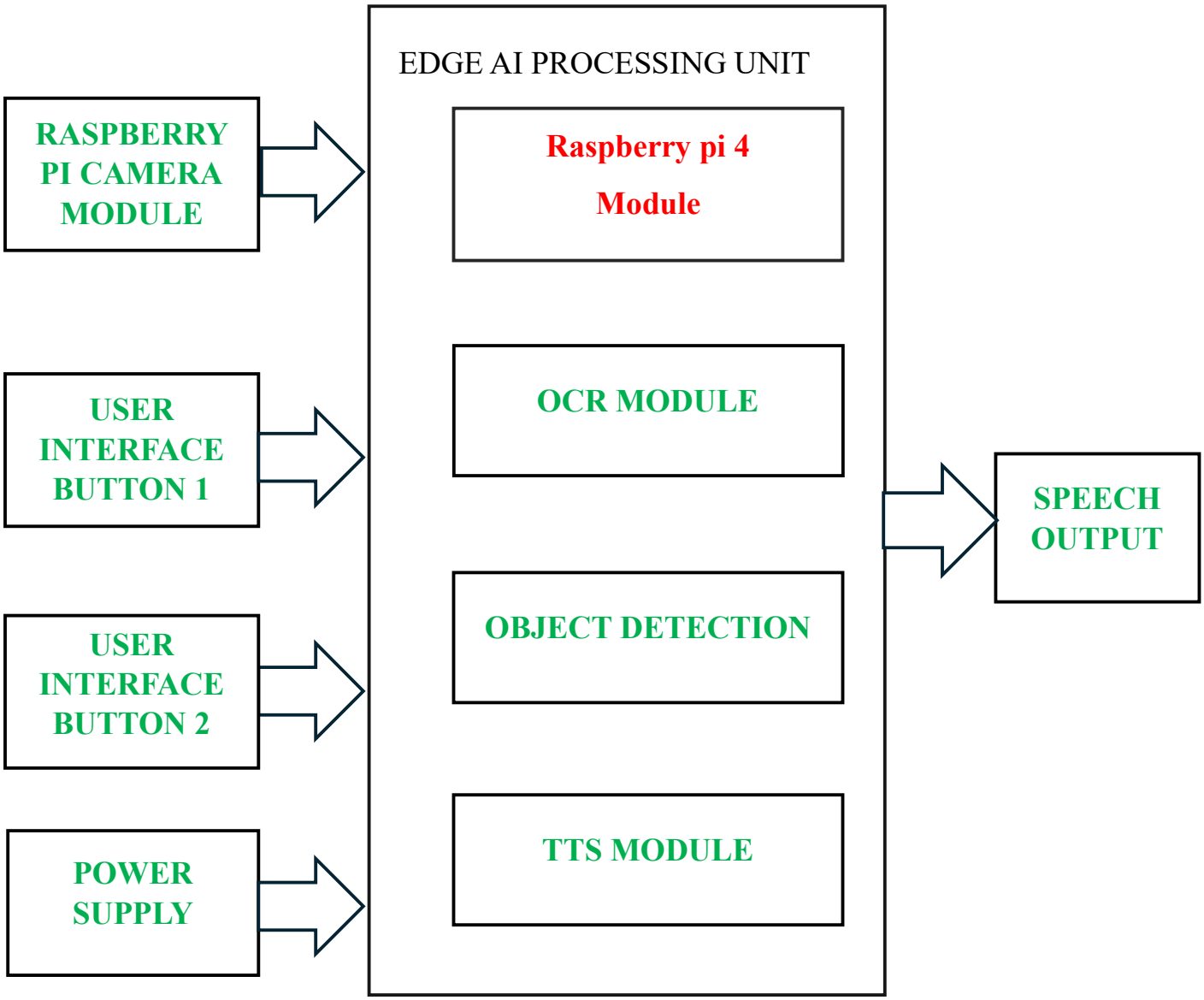

Figure 3.1 Block Diagram

3.2 CIRCUIT DIAGRAM:

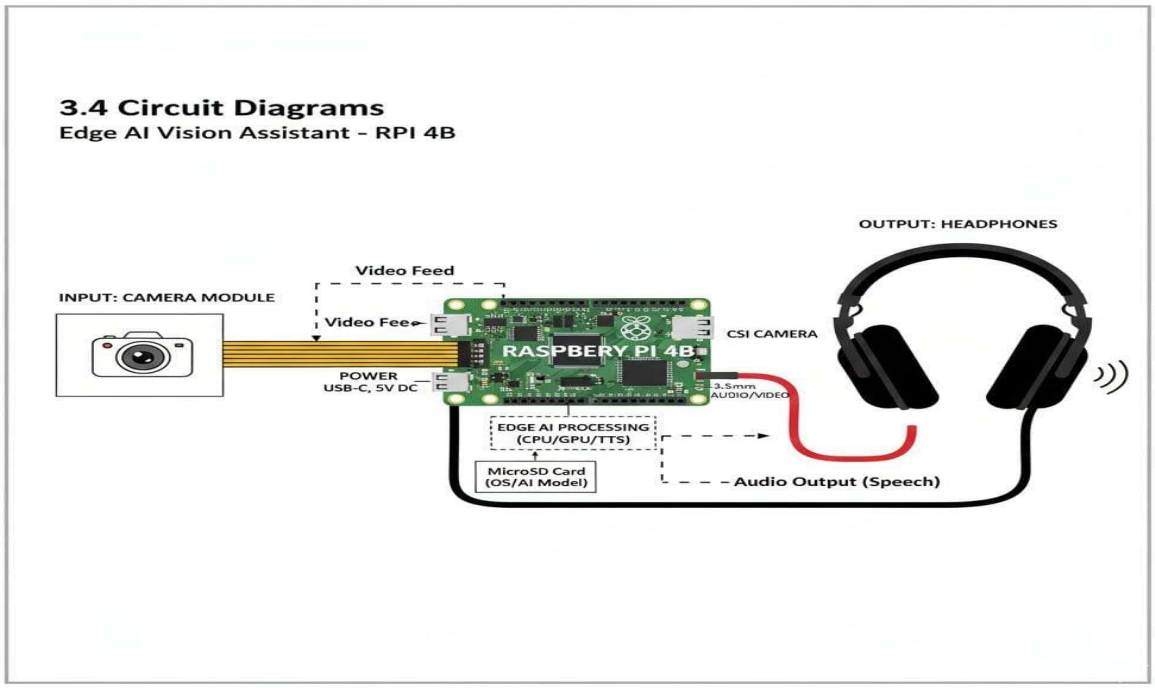

Figure 3.2 Circuit Diagram

3.3 WORKING

The **Edge AI Enabled Smart Vision Assistant for Visually Challenged Individuals** is a practical and intelligently designed system aimed at improving accessibility, independence, and overall situational awareness for people with visual impairments. The system brings together several core AI-based functions such as **Optical Character Recognition (OCR)**, **Text-to-Speech (TTS)**, and **Object Detection**, creating a unified tool capable of providing useful real-time information to the user. These combined features allow the device to identify objects in the environment, read printed or written text, and convert the extracted information into clear audio output that the user can easily understand. The entire solution is built to run directly on **edge computing hardware**, allowing it to operate quickly and efficiently without depending on

constant internet access. By processing everything locally, the system delivers fast responses, maintains user privacy, and remains reliable even in areas with limited or no connectivity.

Image Acquisition and Preprocessing:

The working of the system begins with image acquisition using a high-resolution camera module mounted on wearable equipment, such as smart glasses or a handheld device. The camera continuously captures the surrounding environment in real time. These raw image frames are then pre-processed to enhance clarity and reduce noise. Techniques such as contrast adjustment, brightness normalization, and edge enhancement are applied to ensure optimal image quality for subsequent AI analysis. The preprocessing stage plays a vital role in improving recognition accuracy under varying lighting or environmental conditions.

Optical Character Recognition (OCR):

Once an image containing textual information is captured, the system's OCR module is activated. The OCR algorithm, based on deep learning models like Tesseract, analyzes the frame to detect and extract text from documents, signboards, product labels, or digital screens. The extracted text is then converted into a machine-readable format. This feature allows visually challenged individuals to read printed materials or recognize textual content in their environment instantly. The OCR module is optimized for real-time edge performance, ensuring quick and efficient text extraction without requiring internet connectivity.

Object Detection and Scene Understanding:

In addition to OCR, the system includes an object detection module that scans the user's surroundings and identifies commonly encountered items. This component uses the **MobileNet-SSD** object detection framework, a lightweight and efficient CNN model suitable for real-time processing on edge devices. With this approach, the system can detect a variety of everyday objects such as people, vehicles, furniture, and other obstacles. Once an object is recognized, it is simply labelled and announced to the user, allowing them to gain a clearer understanding of what

is present in their environment. By providing reliable object identification, the system enhances situational awareness and supports more confident and independent movement.

Text-to-Speech (TTS) Conversion:

After text or object data is processed, the TTS module converts this information into natural-sounding audio output. The synthesized speech is generated in a clear, human-like voice that communicates essential details to the user. This includes reading extracted text aloud, describing identified objects. The TTS system uses neural speech synthesis models that ensure smooth and intelligible speech output.

Integrated Operational Workflow:

The process begins with image acquisition and preprocessing, where the camera captures live visuals and enhances the image quality for accurate analysis. Next, the OCR module extracts and digitizes any visible text, allowing the system to interpret written information. At the same time, the object detection module identifies and classifies objects present in the environment, helping the user understand what surrounds them. The extracted text and detected objects are then converted into clear audio output through the TTS engine, enabling the user to receive information in an accessible format. This integrated workflow ensures fast, real-time, and seamless support for visually challenged individuals.

3.4 FLOW CHART

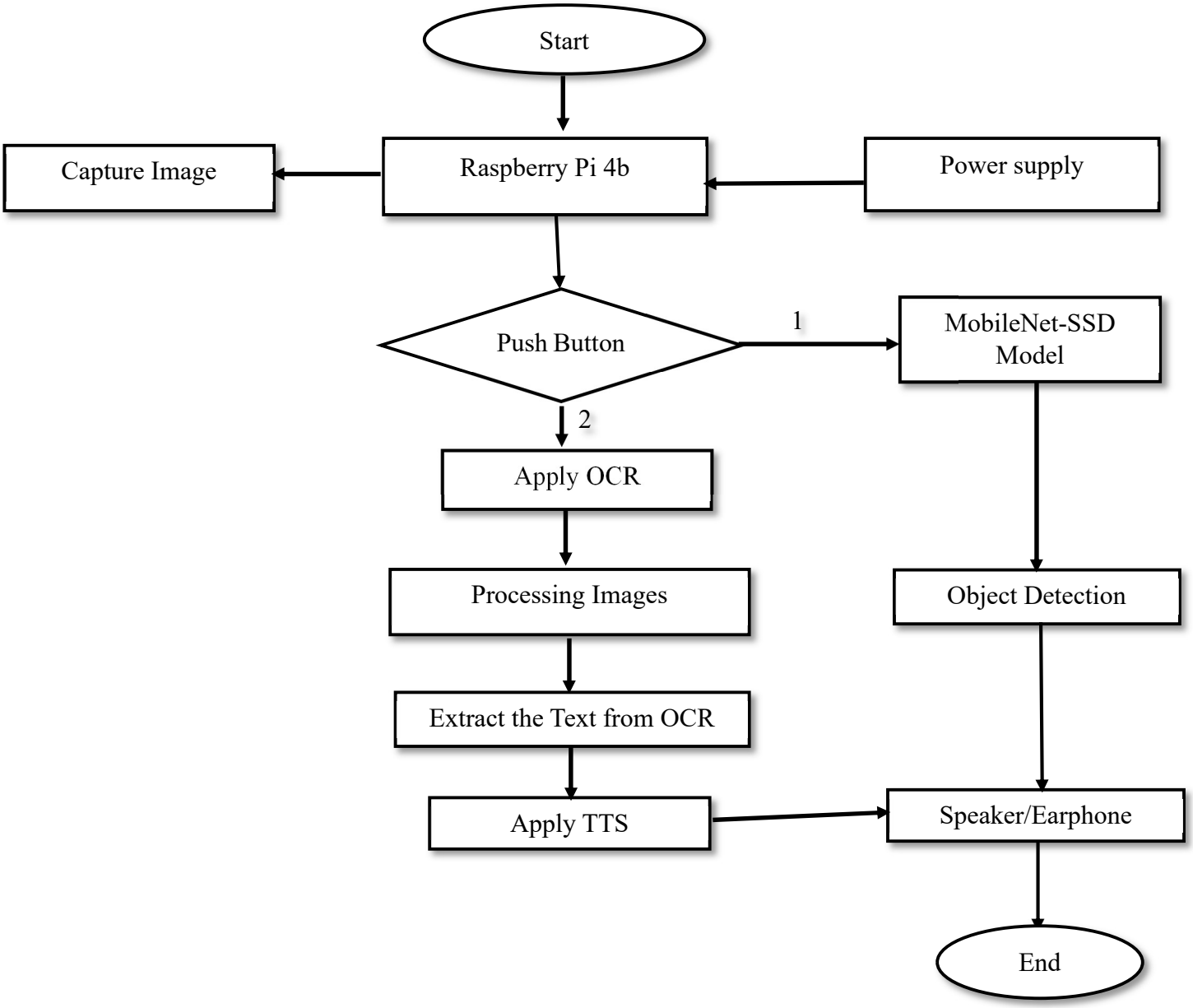

Figure 3.4 Flow Chart

3.5 COMPONENTS REQUIRED

3.5.1 THE RASPBERRY PI 4 MODEL:

Figure 3.5.1 The Raspberry Pi 4 Model

The Raspberry Pi 4 Model B with 4GB of RAM acts as the central brain in home automation and IoT projects, particularly those involving relay modules to control high-power devices like lights, fans, and household appliances. It is a compact, fully functional computer powered by a 1.5 GHz quad-core Cortex-A72 processor and 4GB of memory, making it capable of handling multiple tasks, processing sensor data, and running automation programs simultaneously. In a project with a Two-Channel Relay Module, the Raspberry Pi connects directly to the module through its 40-pin GPIO header. Specific GPIO pins are configured as outputs to control the relays. When the Raspberry Pi sends a signal to the relay input pins, the corresponding relay switches, changing the connection from Normally Closed (NC) to Normally Open (NO) or vice versa, which in turn powers or shuts off the connected device. This switching can be controlled using programming languages like Python or C, allowing the system to respond automatically to sensor readings, scheduled events, or remote commands.

The Raspberry Pi 4 also provides multiple connectivity options, including Wi-Fi, Bluetooth, and Ethernet, which allow it to communicate with smartphones, cloud services, or other devices. This makes remote monitoring and control possible, so lights can be turned on, fans can be activated, or alarms can be triggered from anywhere. Sensors for temperature, humidity, or motion can feed data to the Raspberry Pi, which processes it and activates relays according to predefined rules, such as turning on a fan when the room gets too warm or triggering a security alarm if motion is

detected. The device supports dual monitors through micro-HDMI ports, which can display dashboards or system status, while USB ports allow connection to storage devices for logs or program backups. The microSD card acts as the main storage for the operating system and software, giving flexibility to update or expand automation routines easily.

Importantly, when used with opto-isolated relays, the Raspberry Pi keeps low-voltage control circuits separate from high-voltage devices, ensuring safety for both the user and the electronics. It can also integrate with IoT protocols, APIs, and smart home frameworks, enabling advanced features like app-based or voice-controlled automation. Essentially, the Raspberry Pi 4 serves as the intelligent controller in the system, coordinating sensors, relays, and software logic to create a responsive, reliable, and scalable automation platform. Its small size, energy efficiency, and compatibility with a wide range of sensors and modules make it an ideal choice for projects where high-power devices need to be controlled safely and intelligently. With the Raspberry Pi 4 at the core, users can build a system that combines real-time control, remote access, and smart automation, turning a simple collection of relays and devices into a fully functional, connected home or industrial system.

Working Principle of The Raspberry Pi 4 Model:

In our project, the Raspberry Pi 4 works as the main brain of the system that controls and manages all the connected components. Once power is supplied, it boots the operating system from the microSD card and automatically runs the program written for the project. The Raspberry Pi continuously collects information from connected sensors and the camera module through its GPIO pins and dedicated interfaces. It analyzes this information in real time and makes decisions based on the instructions written in the program. For example, when a specific condition is detected, such as movement captured by the camera or input from a sensor, the Raspberry Pi sends signals to the relay module to switch electrical devices like lights, fans, or alarms ON or OFF.

At the same time, the Raspberry Pi uses its built-in Wi-Fi or Ethernet connection to communicate with other devices, making it possible to monitor or control the system remotely. It stores important data such as logs, images, or videos on the microSD card so that information can be reviewed later. The Raspberry Pi can also send audio alerts through connected speakers or earphones to notify the user about important events.

3.5.2 THE 5MP RASPBERRY PI CAMERA MODULE:

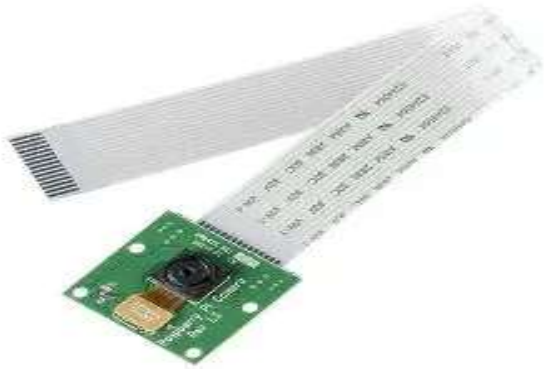

Figure 3.5.2 5mp Raspberry Pi Camera

The **5MP Raspberry Pi Camera Module** for Pi 4 Model B – Rev 1.3 is a compact and versatile imaging device designed to interface directly with Raspberry Pi boards via the dedicated CSI port, enabling high-speed, low-latency communication for real-time image capture and video processing. Equipped with a 5-megapixel OmniVision OV5647 sensor, it is capable of capturing high-quality still images and recording full HD 1080p video at 30 frames per second, making it suitable for a wide range of applications, including home security, robotics, time-lapse photography, wildlife monitoring, and computer vision projects. The module is powered entirely through the Raspberry Pi, eliminating the need for an external power source, and its small form factor allows easy mounting in tight spaces or integration into custom enclosures, robotic systems, or automation setups. In a practical project, the camera can be controlled using Python libraries such as picamera or other supported programming languages, allowing the Raspberry Pi to capture images, record video, or even stream live feeds over a network. This functionality can be integrated with sensor input or relay modules, enabling automation scenarios like motion detection, object recognition, or remote monitoring of devices, appliances, or security zones. The module supports adjustable focus through compatible lenses, allowing flexibility for capturing both close-up and distant subjects, and it performs effectively in low-light conditions, especially when paired with infrared lighting for night-vision applications. When combined with the processing power of the Raspberry Pi 3 or 4, the camera enables advanced features such as real-time image processing, AI-based object detection, and computer vision applications using frameworks like OpenCV or TensorFlow. Its lightweight and energy-efficient design allows

continuous operation without affecting the overall performance of the Raspberry Pi, making it ideal for long-term monitoring, security surveillance, or automated recording tasks. Overall, the 5MP Raspberry Pi Camera Module Rev 1.3 acts as a high-quality visual sensor that bridges the gap between hardware and software, allowing users to capture, process, and analyse images and videos efficiently. Its compatibility with a wide range of programming libraries and the Raspberry Pi's processing capabilities make it an essential component for DIY electronics, robotics, smart home automation, security systems, and educational projects, providing flexibility, reliability, and high performance in a small and easy-to-use package.

Working Principle of the 5MP Raspberry Pi Camera Module:

In our project, the 5MP Raspberry Pi Camera Module works as the visual sensing unit that captures images and video to help the system monitor its surroundings. When the Raspberry Pi is powered on, the camera module is activated through the CSI (Camera Serial Interface) port and becomes ready to capture visual data. It continuously or periodically captures frames based on the program written in the Raspberry Pi. These images or video frames are then sent to the Raspberry Pi for processing in real time. The module converts light from the environment into digital signals using its 5-megapixel image sensor, allowing the system to clearly "see" objects, movement, or changes in the environment.

When the project requires motion detection or monitoring, the camera constantly observes the area and the Raspberry Pi analyzes the captured frames to detect changes or movement. If motion or a specific event is detected, the system can trigger actions such as switching a relay, activating an alarm, turning on a light, or sending notifications. The camera module can also store captured images or recorded videos on the microSD card for later viewing or evidence purposes. In low-light conditions, the camera can be supported with additional lighting to improve image clarity. Overall, the 5MP Raspberry Pi Camera Module works by sensing visual information, converting it into digital data, and sending it to the Raspberry Pi, enabling the project to perform real-time monitoring, detection, and automated responses based on visual input.

3.5.3 RASPBERRY PI MICROSD CARD (32GB):

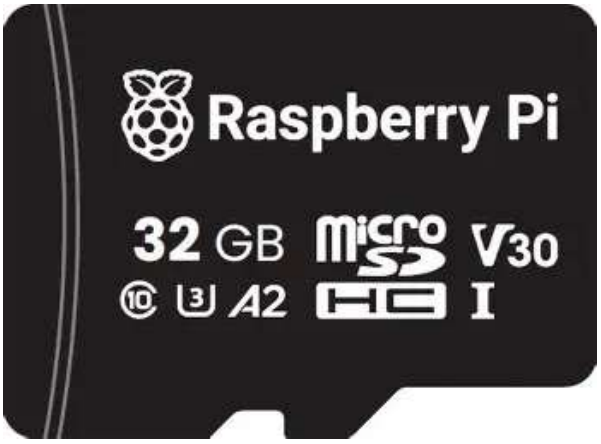

Figure 3.5.3 Raspberry Pi Micro SD Card

A **32GB MicroSD** card serves as the primary storage and boot medium for the Raspberry Pi, providing both the operating system and space for project files, applications, and data storage. It connects directly to the Raspberry Pi via the built-in microSD card slot, allowing the board to read and write data at high speeds depending on the card's class and specifications, which is essential for smooth performance in tasks such as running the operating system, storing logs, or handling media files. In a project involving relay modules and sensors, the MicroSD card stores the operating system—typically Raspberry Pi OS or other Linux-based distributions—as well as any custom scripts or software used to control devices, monitor sensors, or process inputs from connected peripherals like the 5MP camera or relay boards. The card's 32GB capacity provides ample space for storing multiple automation scripts, recorded videos, images captured by the camera, or data logs from sensors, enabling long-term monitoring and project expansion without the need for frequent data offloading. Its removable and portable design allows users to easily back up, clone, or replace the operating system and data, which is particularly useful during testing, updates, or migration to new hardware. High-quality MicroSD cards also support fast read and write speeds, ensuring quick boot times, efficient data access, and smooth execution of programs, which is crucial for real-time applications like motion detection, video recording, or automated control of high-power devices through relay modules. Additionally, the card's durability and reliability make it suitable for continuous operation in automation and IoT projects, where the Raspberry Pi may run 24/7. Overall, the 32GB MicroSD card acts as the backbone of the Raspberry

Pi project's storage and software environment, providing reliable, flexible, and high-performance memory that enables seamless execution of operating systems, automation routines, data logging, and multimedia handling, making it an indispensable component in any Raspberry Pi-based home automation, robotics, or monitoring system.

Working Principle of Raspberry Pi MicroSD Card (32GB):

In our project, the 32GB MicroSD card works as the main storage and boot device for the Raspberry Pi. When power is supplied to the Raspberry Pi, it first looks for the operating system on the MicroSD card and loads it into memory so that the board can start functioning. The card stores the operating system, program files, and all the software needed to run the project. As the project operates, the Raspberry Pi continuously reads data from the MicroSD card and writes new data to it, such as system logs, captured images and videos from the camera module, and saved sensor information. The MicroSD card also allows easy updating of the system, since programs and settings can be modified by simply accessing or re-flashing the card. Because of its relatively large storage capacity, it can store large amounts of data without running out of space quickly. This makes it especially useful for projects that require long-term data storage, like security monitoring or automation logging. Overall, the MicroSD card works as the memory backbone of the system, ensuring that the Raspberry Pi can boot properly, store important data, and run the project smoothly and reliably.

3.5.4 SPEAKER / EARPHONES:

Figure 3.5.4 Speaker / Earphones

A **speaker** and **earphones** are audio output devices used to convert electrical audio signals into sound that people can hear. A speaker is usually designed to produce sound that can be heard openly in a room, making it suitable for music, movies, presentations, or public audio systems. It uses a driver unit that vibrates to create sound waves when an electrical signal passes through it. **Earphones**, on the other hand, are small, personal audio devices that fit directly into or over the ears, allowing private listening without disturbing others.

Both speakers and earphones can be wired or wireless and are commonly used with phones, computers, televisions, and music players. They vary in size, sound quality, and design, and some models include features such as volume control, built-in microphones, noise cancellation, and Bluetooth connectivity. These devices are widely used for entertainment, communication, education, and professional audio work.

Working Principle of Speaker/Earphone:

A speaker or earphones in the project act as the primary audio output component, helping the system communicate with the user through sound rather than visuals alone. They work by receiving electrical audio signals from the Raspberry Pi's audio interface and converting them into audible sound waves that humans can hear clearly. In practical use, this allows the system to produce alert tones, voice messages, confirmation sounds, and warning alarms that inform the user about different system states, such as successful command execution, detection of motion through the camera module, or the activation of relay-controlled devices. These audio outputs can be programmed using software libraries so that specific sounds play automatically based on real-time events, scheduled timers, or user inputs, making the overall system more interactive and intelligent. Speakers are especially useful in environments where visual monitoring is not always possible, as they provide instant awareness through sound, while earphones are ideal for private monitoring and debugging during development or testing phases. In addition, audio feedback improves safety and reliability, particularly in security or automation projects, because critical alerts can be heard even from a distance. The integration of speakers or earphones also enhances user experience by making the system feel more responsive and human-friendly, turning technical system actions into understandable audio cues.

3.5.5 POWER BANK:

Figure 3.5.5 Power Bank

A **power bank** is a portable device used to store electrical energy and supply it to electronic devices whenever needed. It contains a rechargeable lithium-ion or lithium-polymer battery that is charged in advance using a wall charger or computer. Once charged, it can be connected to devices such as smartphones, tablets, wireless earphones, or cameras through a USB cable to provide power and recharge their batteries. Power banks usually have one or more USB output ports and built-in circuits that regulate voltage and protect devices from overcharging, short circuits, and overheating. They come in different capacities, typically measured in milliampere-hours (mAh), and are widely used when access to a wall socket is not available, making them very useful for travel, outdoor activities, and emergencies.

Working Principle of Power Bank:

In our project, the power bank functions as the portable power source that supplies the required electrical energy to the Raspberry Pi and other connected components. It works by storing electrical energy inside its internal rechargeable lithium-ion battery and delivering this energy through a regulated 5V USB output when connected to the system. When the Raspberry Pi is powered using the power bank, it receives a stable voltage through the USB cable, which allows it to boot the operating system and start running the programmed tasks without interruption. This

makes the entire project portable and independent of direct wall power, which is especially useful for outdoor monitoring, mobile robotics, or temporary installations.

The power bank continuously provides power as long as it has sufficient charge, ensuring smooth and stable operation of the camera module, relay circuits, and other peripherals connected to the Raspberry Pi. Many power banks also include built-in protection circuits to prevent overcharging, short circuits, and overheating, which improves the overall safety and reliability of the project. When the battery level becomes low, the power bank can be easily recharged and reused, allowing repeated and long-term use of the system. Overall, the power bank plays a crucial role in making the project flexible and portable by providing a reliable and safe power supply, ensuring that all components operate effectively even in locations where direct electrical outlets are not available.

CHAPTER 4

CODE

4.1 CODE

```
from picamera2 import Picamera2
import cv2
import time
import os
import sys
import subprocess
import pytesseract
from collections import deque, defaultdict
import json
import tempfile

# ---------- MODEL CONFIG ----------
MODEL = "models/MobileNetSSD_deploy.caffemodel"
PROTOTXT = "models/MobileNetSSD_deploy.prototxt"

CLASSES = [
    "background","aeroplane","bicycle","bird","boat","bottle",
    "bus","car","cat","chair","cow","diningtable","dog","horse",
    "motorbike","person","pottedplant","sheep","sofa","train","tvmonitor"
]

# ---------- DEBUG / TUNING ----------
DEBUG = True
CONF_THRESH = 0.45
NMS_THRESH = 0.35
MIN_BOX_WH = 20

# ---------- TEMPORAL CONFIRMATION (optional) ----------
CONFIRM_FRAMES = 1
label_buffers = defaultdict(lambda: deque(maxlen=CONFIRM_FRAMES))

# ---------- LANGUAGE & TTS SETTINGS ----------
OCR_LANG  = "kan+eng"     # Kannada + English OCR

# ---------- ELEVENLABS TTS CONFIG ----------
```

```
ELEVEN_API_KEY=
"9e745c91bf74aaafee862fe29d178cbadc2e5f17769c5b69d1f3f8e16bad8ead"
VOICE_ID_KN = "JkPlDBRtY3CdpXB74wL3"
VOICE_ID_EN = "LQMC3j3fn1LA9ZhI4o8g"
ELEVEN_TIMEOUT = 18  # seconds
ELEVEN_RETRIES = 2
def _espeak_fallback(text: str):
    try:
        subprocess.run(["espeak", "-s", "130", "-v", "en", text], check=False)
    except Exception:
        pass

def speak(text: str, lang: str = "kn"):
    """
    Auto-language ElevenLabs TTS (keeps signature speak(text, lang='kn')).
    If ElevenLabs fails, falls back to espeak so the system remains usable.
    """
    if not text:
        return
    t = str(text).strip()
    # Determine language: use override if provided, else auto-detect Kannada chars
    chosen_lang = "kn" if (lang and str(lang).startswith("kn")) or any(0x0C80 <= ord(c) <= 0x0CFF
for c in t) else "en"
    voice_id = VOICE_ID_KN if chosen_lang == "kn" else VOICE_ID_EN

    # Basic checks
    if not ELEVEN_API_KEY or not voice_id:
        if DEBUG:
            print("ElevenLabs key/voice missing; using espeak fallback.")
        _espeak_fallback(t)
        return

    payload = json.dumps({"text": t})
    for attempt in range(1, ELEVEN_RETRIES + 1):
        tmp_path = None
        try:
            with tempfile.NamedTemporaryFile(delete=False, suffix=".mp3") as tmpf:
                tmp_path = tmpf.name
            curl_cmd = [
                "curl", "--fail", "-s", "-X", "POST",
                f"https://api.elevenlabs.io/v1/text-to-speech/{voice_id}",
```

```
            "-H", f"xi-api-key: {ELEVEN_API_KEY}",
            "-H", "Content-Type: application/json",
            "-d", payload,
            "--output", tmp_path,
            "--connect-timeout", "8", "--max-time", str(ELEVEN_TIMEOUT)
        ]
        proc = subprocess.run(curl_cmd, stdout=subprocess.DEVNULL, stderr=subprocess.PIPE)
        if proc.returncode != 0:
            if DEBUG:
                print("ElevenLabs curl failed:", proc.stderr.decode(errors="ignore"))
            if attempt == ELEVEN_RETRIES:
                _espeak_fallback(t)
                return
            time.sleep(0.5)
            continue
        if not os.path.isfile(tmp_path) or os.path.getsize(tmp_path) < 200:
            if DEBUG:
                print("ElevenLabs produced invalid/empty audio, size:", os.path.getsize(tmp_path) if
tmp_path and os.path.isfile(tmp_path) else "no-file")
            if attempt == ELEVEN_RETRIES:
                _espeak_fallback(t)
                return
            time.sleep(0.5)
            continue
        # play via mpg123 (system must have mpg123 installed)
        play_proc = subprocess.run(["mpg123", "-q", tmp_path], check=False)
        # done
        return
    except subprocess.TimeoutExpired:
        if DEBUG:
            print("ElevenLabs request timed out")
        if attempt == ELEVEN_RETRIES:
            _espeak_fallback(t)
            return
        time.sleep(0.5)
        continue
    except Exception as e:
        if DEBUG:
            print("TTS error:", e)
        if attempt == ELEVEN_RETRIES:
            _espeak_fallback(t)
```

```
            return
        time.sleep(0.5)
        continue
    finally:
        try:
            if tmp_path and os.path.isfile(tmp_path):
                os.remove(tmp_path)
        except Exception:
            pass
# ---------- helper: detect OCR language ----------
def detect_language(text: str) -> str:
    if not text:
        return "en"
    eng = 0
    kan = 0
    for c in text:
        o = ord(c)
        if 0x0C80 <= o <= 0x0CFF:
            kan += 1
        elif c.isascii() and c.isalpha():
            eng += 1
    return "kn" if kan > eng else "en"
# ---------- CHECK MODEL FILES ----------
if not os.path.isfile(MODEL) or not os.path.isfile(PROTOTXT):
    print("ERROR: Model files not found in 'models/'")
    sys.exit(1)

try:
    net = cv2.dnn.readNetFromCaffe(PROTOTXT, MODEL)
except Exception as e:
    print("ERROR loading DNN model:", e)
    sys.exit(1)

net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)

# ---------- SETUP CAMERA ----------
picam2 = Picamera2()
picam2.options["display"] = False
config = picam2.create_preview_configuration(
    main={"size": (640, 480), "format": "RGB888"}
```

```
)
picam2.configure(config)
picam2.start()
time.sleep(1.0)

print("\nBlind Assistant Running...")
print("Press 'O' → Speak detected objects")
print("Press 'R' → OCR + Speak text")
print("Press 'Q' → Quit program")
speak("ಸಿಸ್ಟಂ ಸಿದ್ಧವಾಗಿದೆ", lang="kn")  # "System ready"


try:
    while True:
        frame = picam2.capture_array()
        (h, w) = frame.shape[:2]

        # ---------- OBJECT DETECTION PREPARE ----------
        blob = cv2.dnn.blobFromImage(
            cv2.resize(frame, (300, 300)),
            0.007843,
            (300, 300),
            127.5,
            swapRB=True
        )
        net.setInput(blob)
        detections = net.forward()

        # Raw collect
        boxes = []
        confidences = []
        class_ids = []
        if DEBUG:
            print("---- RAW DETS ----")

        for i in range(detections.shape[2]):
            conf = float(detections[0, 0, i, 2])
            idx = int(detections[0, 0, i, 1])
            # calculate box
            box = detections[0, 0, i, 3:7] * [w, h, w, h]
            sx, sy, ex, ey = box.astype("int")
```

```
        width = ex - sx
        height = ey - sy
        if DEBUG:
            label_name = CLASSES[idx] if 0 <= idx < len(CLASSES) else f"idx_{idx}"
            print(f"i={i} id={idx} label={label_name} conf={conf:.3f} box=({sx},{sy},{ex},{ey})
w,h=({width},{height})")
        # collect candidates above a low threshold for NMS
        if conf < 0.25 or width <= 0 or height <= 0:
            continue
        boxes.append([max(0, sx), max(0, sy), width, height])
        confidences.append(conf)
        class_ids.append(idx)

    # Apply NMS
    final_indices = []
    try:
        indices = cv2.dnn.NMSBoxes(boxes, confidences, score_threshold=CONF_THRESH,
nms_threshold=NMS_THRESH)
        if isinstance(indices, (list, tuple)):
            for it in indices:
                if isinstance(it, (list, tuple)) and len(it) > 0:
                    final_indices.append(int(it[0]))
                else:
                    final_indices.append(int(it))
        else:
            for it in indices:
                final_indices.append(int(it[0]) if hasattr(it, '__len__') else int(it))
    except Exception:
        final_indices = [i for i, c in enumerate(confidences) if c >= CONF_THRESH]

    if DEBUG:
        print("---- POST-NMS & FILTER CANDIDATES ----")

    current_frame_labels = set()
    ghost_classes = ["sofa", "tvmonitor", "diningtable", "chair", "bottle"]

    for i in final_indices:
        if i is None or i < 0 or i >= len(class_ids):
            continue
        conf = confidences[i]
        idx = class_ids[i]
```

```
        label = CLASSES[idx] if 0 <= idx < len(CLASSES) else f"idx_{idx}"
        sx, sy, width, height = boxes[i]
        ex = sx + width; ey = sy + height

        if width < MIN_BOX_WH or height < MIN_BOX_WH:
            if DEBUG:
                print(f"skipping small box for {label} w,h=({width},{height})")
            continue

        if label == "person":
            aspect = width / float(height) if height else 0
            if aspect < 0.12 or aspect > 1.6:
                if DEBUG:
                    print(f"skipping person by aspect {aspect:.2f}")
                continue

        current_frame_labels.add(label)
        cv2.rectangle(frame, (sx, sy), (ex, ey), (255, 255, 255), 2)
        cv2.putText(frame, f"{label} {conf:.2f}", (sx, sy - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)
        if DEBUG:
            print(f"ACCEPT -> {label} conf={conf:.3f} box=({sx},{sy},{ex},{ey})")

    # update buffers (CONFIRM_FRAMES=1 while debugging)
    for lbl in current_frame_labels:
        label_buffers[lbl].append(1)
    for lbl in list(label_buffers.keys()):
        if lbl not in current_frame_labels:
            label_buffers[lbl].append(0)
        if len(label_buffers[lbl]) == CONFIRM_FRAMES and sum(label_buffers[lbl]) == 0:
            del label_buffers[lbl]
    # display
    cv2.imshow("Blind Assistant (q=quit)", frame)
    key = cv2.waitKey(1) & 0xFF
    # keys
    if key == ord('q'):
        break
    if key == ord('o') or key == ord('O'):
        # confirm immediate labels
        confirmed = []
        for lbl, dq in label_buffers.items():
```

```
            if sum(dq) >= 1:
                confirmed.append(lbl)
        if confirmed:
            for lbl in sorted(confirmed):
                speak(lbl, lang="en")
                time.sleep(0.5)
        else:
            speak("ಯಾವುದೇ ವಸ್ತು ಪತ್ತೆಯಾಗಿಲ್ಲ", lang="kn")
    if key == ord('r') or key == ord('R'):
        gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
        gray = cv2.threshold(
            gray, 0, 255,
            cv2.THRESH_BINARY | cv2.THRESH_OTSU
        )[1]
        gray = cv2.medianBlur(gray, 3)
        try:
            text = pytesseract.image_to_string(gray, lang=OCR_LANG)
        except Exception as e:
            print("OCR error:", e)
            speak("ಪಠ್ಯ ಪತ್ತೆದೋಷ", lang="kn")
            text = ""
        clean = " ".join(text.split())
        print("OCR:", clean)
        if clean:
            lang_detected = detect_language(clean)
            speak(clean, lang=lang_detected)
        else:
            speak("ಪಠ್ಯ ಕಾಣಿಸಲಿಲ್ಲ", lang="kn")
finally:
    picam2.stop()
    cv2.destroyAllWindows()
    print("Clean exit")
```

CHAPTER 5

EVALUATION

5.1 ADVANTAGES:

- **Enhanced Independence:** Helps visually challenged individuals navigate their surroundings without constant assistance from others.
- **Real-Time Object Detection:** Uses edge AI to instantly identify obstacles, people, and objects, improving situational awareness.
- **Improved Safety:** Provides immediate audio alerts about nearby hazards, reducing the risk of accidents and injuries.
- **Offline Functionality:** Performs AI processing directly on the device, ensuring reliable operation even without internet connectivity.
- **Fast Response Time:** Edge-based processing reduces latency, providing quicker feedback compared to cloud-dependent systems.
- **Privacy Protection**: Keeps all visual data on the local device, minimizing the risk of data breaches or misuse.
- **Portability:** Compact and lightweight design makes it easy to carry and use throughout the day.
- **User-Friendly Interaction:** Simple audio guidance and voice feedback make the system easy to understand and operate.
- **Scalable Technology:** Can be upgraded with additional AI features such as face recognition, text reading, and navigation assistance.
- **Cost-Effective Solution:** Uses affordable hardware and software, making assistive technology more accessible to a wider community.
- **Customizable User Experience:** The system can be tailored to individual user needs, such as adjusting alert sensitivity, audio language, voice speed, and notification types for greater comfort and effectiveness.

5.2 DISADVANTAGES:

- **Initial Setup Cost**: High-quality cameras and AI-capable processors can increase the overall cost of the system.
- **Limited Battery Life:** Continuous camera use and AI processing can quickly drain the battery, requiring frequent recharging.
- **Environmental Sensitivity**: Performance can be affected by poor lighting, fog, rain, or crowded environments.
- **Limited Field of View:** The camera can only detect objects within its viewing angle, possibly missing obstacles outside that area.
- **Processing Limitations:** Edge devices have limited computational power compared to cloud servers, which may affect the accuracy of complex AI tasks.

5.3 APPLICATIONS:

- **Obstacle Detection and Navigation:** Guides visually challenged individuals by detecting obstacles like stairs, poles, vehicles, and uneven surfaces in real time.
- **Indoor Assistance:** Helps in navigating indoor spaces such as homes, offices, and public buildings by detecting doors, furniture, and hallways.
- **Object and Text Recognition:** Identifies everyday objects, signs, and text (like shop names or instructions) to provide useful contextual information.
- **Facial and Crowd Awareness**: Recognizes familiar faces or alerts users in crowded environments to enhance social interactions and safety.
- **Public Transportation Support:** Assists in identifying buses, train platforms, ticket counters, or boarding areas for independent travel.
- **Enhanced Independence in Daily Tasks:** Supports routine activities such as shopping, reading, or moving around unfamiliar environments safely.

CHAPTER 6

RESULT ANALYSIS

6.1 PROJECT MODEL

Figure 6.1 Project Model

6.2 PROJECT RESULT

6.2.1 OCR TTS

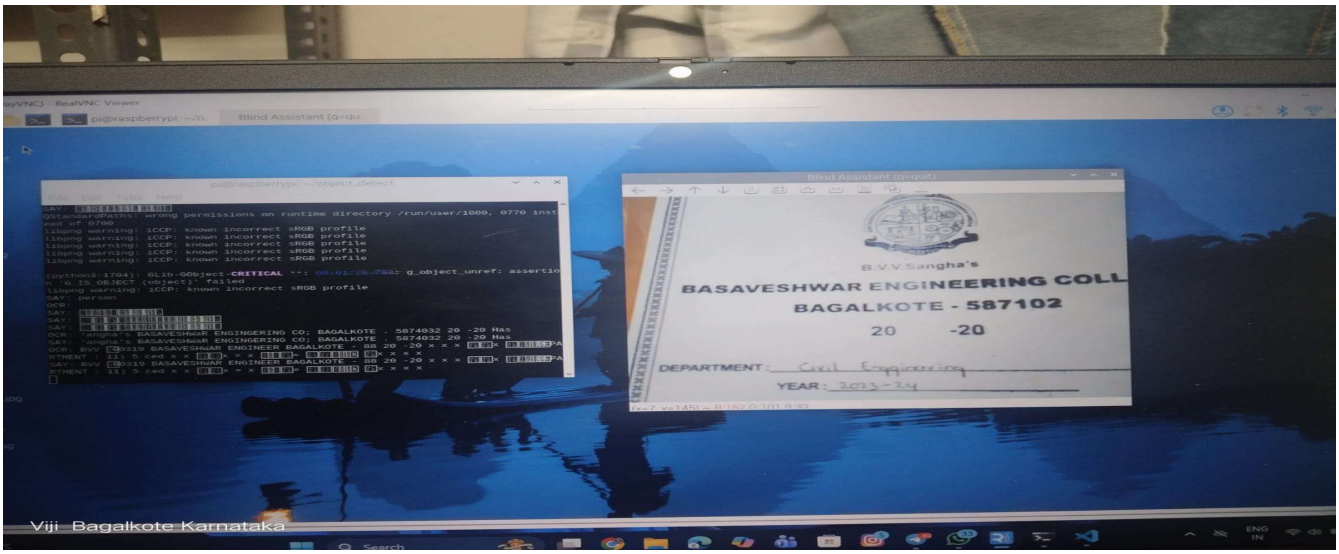

Figure 6.2.1 OCR TTS Result

6.2.2 OBJECT DETECTION

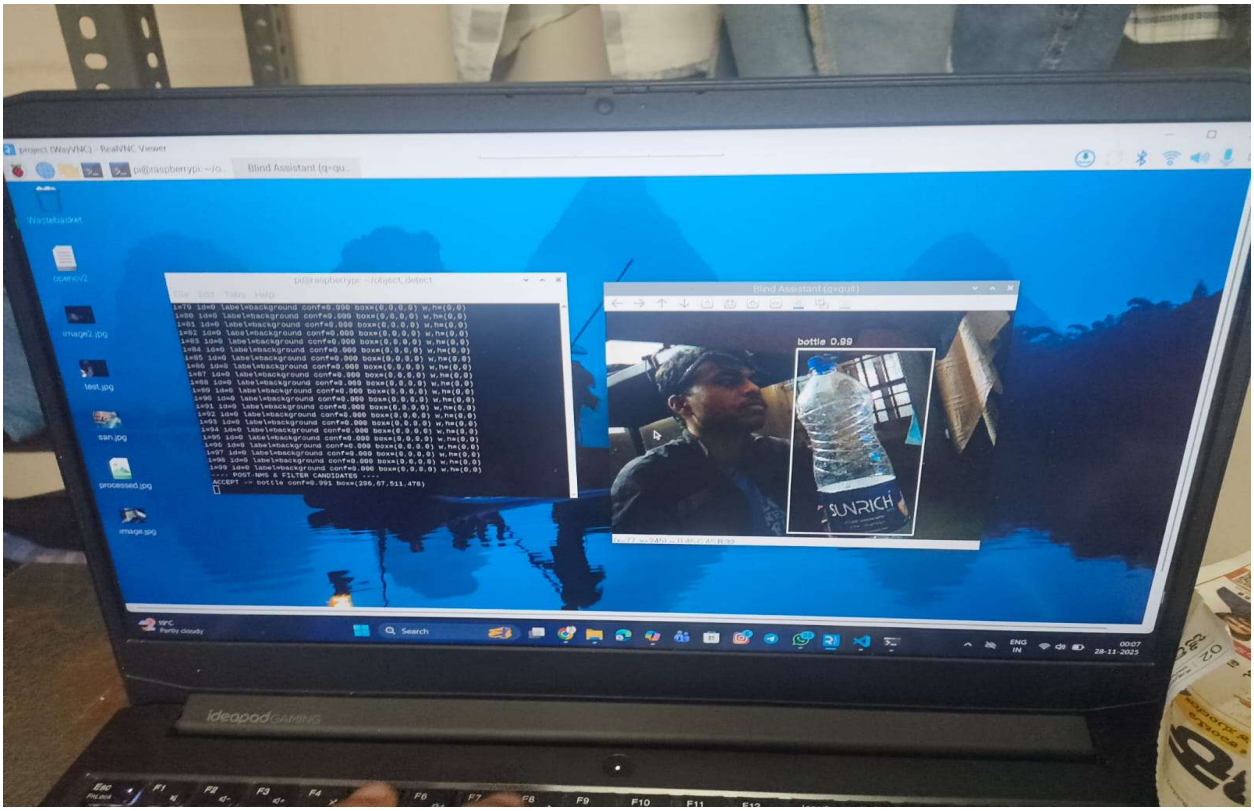

Figure 6.2.2 Object Detection Result

CHAPTER 7

CONCLUSION

The proposed Edge AI Enabled Smart Vision Assistant effectively supports visually impaired individuals by converting visual information into meaningful audio feedback. Through the integration of OCR, the system accurately extracts printed text, enabling hands-free reading of documents, labels, and signage. Real-time object detection using MobileNet-SSD allows users to recognize and avoid obstacles, enhancing spatial awareness and safety. Implemented on a Raspberry Pi, the system is cost-effective, portable, and fully functional offline, making it practical for everyday use. Overall, this project significantly improves the independence, confidence, and quality of life of visually impaired individuals by providing reliable guidance and situational awareness in both indoor and outdoor environments.

REFERENCE

1. R. Himali, S. Adithya, and M. N. Ravishankara, "Assistive Device for the Visually Impaired using Face and Optical Character Recognition with Voice Activation," *IJERT*, vol. 14, no. 4, 2025.
2. F. Gao *et al.*, "Wearable Obstacle Avoidance Device for Visually Impaired Individuals with Cross-Modal Learning," *Nature Communications*, vol. 16, no. 1, 2025.
3. S. Tokmurziyev *et al.*, "LLM-Glasses: GenAI-driven Glasses with Haptic Feedback for Navigation of Visually Impaired People," *arXiv preprint arXiv:2503.16475*, 2025.
4. SpeakEasy Reader, "An OCR-Based Text-to-Speech Device for Enhanced Accessibility in Assisting Reading Challenges," 2024.
5. P. Kumar and S. Reddy, "A Novel Real-Time Text-to-Speech System Using Raspberry Pi," *IIETA*, 2024.
6. A. Patel, "Converting Image Text to Speech Using Raspberry Pi," *JISEM+*, vol. 1, 2023.
7. D. Zhao *et al.*, "Embedded System Based Raspberry Pi 4 for Text Detection and Recognition," *TechScience Journal*, 2023.
8. S. Banerjee *et al.*, "Enhancing Accessibility for Visually Impaired via Wearable OCR + TTS on Raspberry-Pi Ring Device," *SSRN*, 2024.
9. L. Wang *et al.*, "Improving ASR Performance with OCR Through Word Frequency Difference," *ResearchGate preprint*, 2025.
10. Y. Zhang and H. Liu, "MathReader: Text-to-Speech for Mathematical Documents," *arXiv preprint arXiv:2401.16202*, 2025.
11. T. Li *et al.*, "Towards Controllable Speech Synthesis in the Era of Large Language Models: A Survey," *arXiv preprint arXiv:2405.08432*, 2024.
12. K. Jain and A. Bhatia, "Image Processing Based on OCR with Text to Speech for Visually Impaired," *JSCER*, 2023.
13. R. Mishra *et al.*, "AI-WEAR: Smart Text Reader for Blind/Visually Impaired," *IJARCS*, 2023.
14. V. Kumar *et al.*, "Real-Time Object Detection using YOLOv5 on Edge Devices," *IEEE Access*, vol. 12, pp. 45120–45133, 2024.